

GA/ES Manual Operation Mastery Workbook

Representation -> Operator -> Output -> MCQ Answer

This is the dedicated manual-operation workbook. It is not a broad theory textbook. It is a practical reference for the MCQ patterns that ask you to identify, apply, calculate, validate, and distinguish operations in Genetic Algorithms (GA) and Evolution Strategies (ES).

The workbook is built for the questions that look like:

- "Given these parents, what offspring result from PMX / OX / CX / ECX / uniform crossover?"
- "Given this before/after chromosome, what mutation was applied?"
- "Which mutation/crossover is valid for binary / integer / real / permutation representation?"
- "Which selection method is closest to the probability distribution?"
- "Which step acts on current population, parents, offspring, or current+offspring?"
- "Within ES, which statements about mutation, recombination, representation, and selection are true?"

How to use this workbook

Use this document in three passes.

1. **Compatibility pass.** Learn which operations belong to each representation.
2. **Manual execution pass.** Work each operator card: input, steps, output, validation.
3. **MCQ pass.** Use the practice questions and answer keys to drill fast recognition.

The core exam rule

Every operation question should be solved by this chain:

Identify family -> identify representation -> identify operator -> apply exact procedure -> validate output -> eliminate fake options

Do not guess from memory when arrays are given. Run the procedure.

Part 0 - Master Control Tables

0.1 GA vs ES master differential table

Component	Genetic Algorithm (GA)	Evolution Strategies (ES)	Main MCQ trap
Main role	General population-based evolutionary search	Continuous parameter optimization, mutation-centered	Applying GA properties to ES
Classical representation	Binary strings; also integer, real, permutation	Real-valued vectors, often with strategy parameters	Saying ES is binary/canonical GA
Main operator	Recombination/crossover, especially in canonical GA	Mutation, usually probability 1	Low mutation probability in ES
Mutation	Secondary in canonical GA; usually low probability	Central operator; Gaussian/non-uniform; probability 1	Confusing GA low mutation with ES mutation
Recombination	Important/high probability;	Secondary; local/global,	PMX/OX/CX/ECX are GA

	operator depends on representation	discrete/intermediate	permutation operators, not ES core
Parent selection	Fitness-based methods: FPS, SUS, tournament, rank	Often random/uniform in course framing	FPS as ES property
Survivor selection	Generational, steady-state, elitism, age/quality, GENITOR	(mu,lambda) or (mu+lambda)	Roulette survivor selection in ES
Self-adaptation	Not canonical	Central: strategy parameter sigma can evolve	Missing sigma logic
Typical MCQ clue	binary + FPS + low mutation + high recombination	real vector + sigma + mutation probability 1	Mixing cues

0.2 Representation -> operation compatibility matrix

Representation	Valid mutation	Valid crossover/recombination	Invalid or risky traps
Binary	bit flip / negation	one-point, multi-point, uniform	PMX/OX/CX/ECX, arithmetic without decoding
Integer	creep/fluaj, random reset, bounded reset	one-point, multi-point, uniform, arithmetic with rounding/repair	claiming arithmetic is always forbidden
Real	uniform mutation, non-uniform mutation, Gaussian/local mutation	arithmetic, blend, one-point, multi-point, uniform	swap/insertion/inversion/PMX/OX/CX/ECX
Permutation	swap, insertion, inversion, scramble/amestec	PMX, OX/OCX, CX, ECX	raw arithmetic; raw one-point unless repaired
ES real vector	Gaussian/non-uniform mutation, self-adaptive sigma	local/global, discrete/intermediate recombination	canonical binary GA, PMX/OX/CX/ECX

0.3 Which data object does each operation use?

Operation	Input	Output	Fast mental command
Initialization	search space + representation	initial population	before evolution starts
Fitness/evaluation	one candidate/individual	fitness score	evaluates candidate, not algorithm
Parent selection	current evaluated population	parent pool / mating pool	who reproduces?
Recombination/crossover	selected parents	offspring/descendants	combines parents
Mutation	offspring/descendants, sometimes candidate	mutated offspring/candidate	changes genes
Survivor selection	current population and/or offspring	next generation	who remains?
Termination	run state	stop or continue	mandatory stopping rule

0.4 The operator recognition cheat table

MCQ clue	Think immediately
----------	-------------------

"mutation probability 1"	ES mutation
"low mutation probability"	canonical GA
"binary + FPS + high recombination"	canonical GA
"permutation-type chromosomes"	PMX/OX/CX/ECX or permutation mutation
"positions i and j" with two parents	crossover with cut points
"mapped / partially mapped"	PMX
"relative order" or "order of events"	OX / OCX
"cycle" or "absolute allele positions"	CX
"edges" or "common relationships"	ECX
"exactly two positions exchanged"	swap mutation
"segment reversed"	inversion mutation
"segment shuffled"	scramble/amestec mutation
"one element moved and block shifted"	insertion mutation
"small integer change"	creep/fluj
"closest to probability distribution"	SUS
"random subset, best wins"	tournament selection
"preserve best individual"	elitism
"select from offspring only"	(μ , λ) ES
"select from parents + offspring"	(μ + λ) ES

Part I - Representation Pages: What can be applied to what?

1. Binary representation

Theory

Binary representation encodes a chromosome as a string of 0s and 1s. It is the classical/canonical GA representation. It is used when each gene is yes/no, selected/not selected, active/inactive, or true/false.

Valid operations

Operation class	Valid binary operations
Mutation	bit flip / negation
Crossover	one-point, multi-point, uniform
Parent selection	FPS, SUS, tournament, rank, etc.
Survivor selection	all GA survivor models

--	--

Invalid/risky operations

PMX, OX, CX, ECX, swap, insertion, inversion, and scramble are not binary operators unless the binary string is first decoded into a permutation. Do not apply permutation logic to a raw bit string.

Manual example: bit flip

Input chromosome:

X = 1 0 1 1 0 0 1

Mutation positions: 2 and 5.

Steps:

1. Position 2 is 0, flip to 1.
2. Position 5 is 0, flip to 1.
3. All other positions remain unchanged.

Output:

X' = 1 1 1 1 1 0 1

Manual example: one-point crossover

Parents:

P1 = 1 1 0 1 | 0 0 1 0
P2 = 0 0 1 0 | 1 1 0 1

Cut after position 4.

Steps:

1. Child 1 takes prefix from P1 and suffix from P2.
2. Child 2 takes prefix from P2 and suffix from P1.

Outputs:

C1 = 1 1 0 1 1 1 0 1
C2 = 0 0 1 0 0 0 1 0

Binary MCQ traps

Trap	Why wrong
Apply PMX to binary	PMX requires permutation uniqueness
Apply arithmetic without decoding	arithmetic is numeric, not bit-level canonical crossover
Say bit flip changes many/all bits automatically	mutation probability controls which bits flip
Say binary always guarantees optimum	representation does not guarantee optimality

2. Integer representation

Theory

Integer representation uses genes that can take integer values, often more than two possible values. It is used for discrete choices, counts, assignments, category codes, or encoded directions.

Valid operations

Operation class	Valid integer operations
Mutation	creep/fluaj, random reset, bounded reset
Crossover	one-point, multi-point, uniform, arithmetic with rounding/repair
Validation	check domain bounds and integer values

Manual example: creep/fluaj mutation

Input:

```
X = [4, 7, 2, 9, 5]
```

Creep mutation at position 2 with step -1.

Steps:

1. Identify gene at position 2: value 7.
2. Add small integer step: $7 + (-1) = 6$.
3. Check bounds. If allowed, keep 6.

Output:

```
X' = [4, 6, 2, 9, 5]
```

Manual example: random reset

Input:

```
X = [4, 7, 2, 9, 5]
```

Reset position 4 to a new valid integer 3.

Output:

```
X' = [4, 7, 2, 3, 5]
```

Manual example: arithmetic recombination with integer repair

Parents:

```
P1 = [10, 20, 30]
P2 = [20, 10, 40]
alpha = 0.3
```

Formula:

```
C1 = alpha*P1 + (1-alpha)*P2
```

Steps:

1. Gene 1: $0.310 + 0.720 = 17$.
2. Gene 2: $0.320 + 0.710 = 13$.
3. Gene 3: $0.330 + 0.740 = 37$.
4. Already integers, so no rounding needed.

Output:

C1 = [17, 13, 37]

If results are decimal, round according to course rule or nearest integer, then check constraints.

Integer MCQ traps

Trap	Correction
"Arithmetic crossover impossible for integers"	It is possible with rounding/repair; some keys may be inconsistent
Apply permutation mutation to integer vector	swap can exchange integer positions, but it is not the usual integer mutation unless representation is order-based
Treat creep as random reset	creep is small local numeric change; reset is arbitrary valid replacement

3. Real-valued representation

Theory

Real representation uses continuous values. It is common in continuous optimization and also central in ES.

Valid operations

Operation class	Valid real operations
Mutation	uniform, non-uniform, Gaussian/local perturbation
Recombination	arithmetic/blend, one-point, multi-point, uniform
Validation	check bounds, continuity, constraints

Manual example: Gaussian mutation

Input:

X = [2.0, 5.0, 1.0]
sigma = 0.5
z = [0.3, -0.7, 1.2]

Formula:

$X' = X + \text{sigma} * z$

Steps:

1. Gene 1: $2.0 + 0.5 * 0.3 = 2.15$.
2. Gene 2: $5.0 + 0.5 * (-0.7) = 4.65$.
3. Gene 3: $1.0 + 0.5 * 1.2 = 1.60$.

Output:

X' = [2.15, 4.65, 1.60]

Manual example: uniform mutation

Input:

```
X = [2.0, 5.0, 1.0]
Bounds for gene 2: [0, 10]
```

Uniform mutation at gene 2 produces random value 7.4.

Output:

```
X' = [2.0, 7.4, 1.0]
```

Uniform mutation replaces a value with a new value drawn uniformly from an allowed interval. It is not the same as uniform crossover.

Manual example: non-uniform mutation

Input:

```
x_i = 5.0
upper bound = 10
current generation t = 20
max generation T = 100
random factor r = 0.4
```

General idea:

```
x_i' = x_i + Delta(t, upper - x_i)
```

Non-uniform mutation changes less as the generation number increases. The exact formula may vary by course, but the recognition rule is stable: non-uniform mutation is real-valued, generation-dependent, and not a permutation mutation.

Real representation traps

Trap	Correction
Apply PMX/OX/CX/ECX	These require permutations
Treat Gaussian mutation as bit flip	Gaussian adds numeric noise
Confuse uniform mutation with uniform crossover	mutation changes one chromosome; crossover combines parents
Forget bounds	real mutation often needs repair/clamping

4. Permutation representation

Theory

Permutation representation is an ordered list where every allele appears exactly once. It is used for routes, orders, schedules, rankings, and problems where the same item cannot be repeated.

Valid operations

Operation class	Valid permutation operations
Mutation	swap, insertion, inversion, scramble/amestec
Crossover	PMX, OX/OCX, CX, ECX

Validation	same length, same allele set, no duplicates, no missing values

The permutation validity check

Always run this check before accepting an MCQ option.

1. Same length as parents?
2. Same allele set as parents?
3. No duplicates?
4. No missing values?
5. Required segment/order/cycle/edge property preserved?

Permutation traps

Trap	Correction
Raw one-point crossover	often creates duplicates/missing values
Arithmetic crossover	not meaningful for pure permutations
A child with duplicate allele	invalid immediately
A child missing one city/job	invalid immediately
Confusing PMX/OX/CX/ECX	each preserves a different structure

Part II - Parent Selection: how parents are chosen

5. Parent selection overview

Parent selection answers the question: **who reproduces?** It happens after evaluating the current population and before recombination.

Parent selection method	Core idea	Uses probability?	Typical MCQ clue
Fitness proportionate selection (FPS)	probability proportional to fitness	yes	roulette, $p_i = f_i / \text{sum}(f)$
Roulette wheel	spin once per selected parent	yes	roulette arm spun repeatedly
Stochastic universal sampling (SUS)	one spin with equally spaced pointers	yes	closest to probability distribution
Tournament	random subset, best wins	not proportional	random group, best selected
Rank selection	probability based on rank	yes	rank not raw fitness
Sigma scaling	scaled FPS variant	yes	uses mean/std scaling
Windowing	scaled FPS variant	yes	subtract worst or window value

5.1 Fitness proportionate selection (FPS)

Theory

Each individual gets selection probability proportional to its fitness.

Formula:

$$p_i = f_i / \sum(f_j)$$

Step-by-step procedure

1. List each individual and fitness.
2. Compute total fitness.
3. Divide each fitness by total.
4. Build cumulative probability intervals.
5. For each random number r , choose the interval containing r .

Worked example

Population fitness:

Individual	Fitness
A	10
B	30
C	15
D	40
E	5

Total = 100.

Individual	Fitness	Probability	Cumulative interval
A	10	0.10	0.00-0.10
B	30	0.30	0.10-0.40
C	15	0.15	0.40-0.55
D	40	0.40	0.55-0.95
E	5	0.05	0.95-1.00

If $r = 0.72$, choose D.

MCQ traps

Trap	Correction
FPS always preserves best	no, stochastic selection can miss best
FPS cannot select weak individuals	it can, with small probability
FPS is tournament	no, tournament samples a subset
FPS disadvantage	premature convergence if super-individual dominates

5.2 Roulette wheel selection

Roulette selection is the usual implementation of FPS.

Step procedure

1. Convert fitness to probabilities.
2. Create roulette sectors proportional to probabilities.
3. Spin once for each parent needed.
4. Select the sector hit by the spin.

Worked example

Using previous cumulative intervals, if random numbers are:

$r = 0.04, 0.28, 0.51, 0.99$

Selections:

r	Selected
0.04	A
0.28	B
0.51	C
0.99	E

5.3 Stochastic universal sampling (SUS)

Theory

SUS uses one random start and equally spaced pointers. It usually follows the target probability distribution better than repeated independent roulette spins.

Step-by-step procedure

1. Decide how many parents/selections N are needed.
2. Compute spacing = $1/N$.
3. Choose a random start r in $[0, \text{spacing})$.
4. Generate pointers: $r, r+\text{spacing}, r+2\text{spacing}, \dots$
5. Use cumulative intervals to select individuals.

Worked example

Need $N = 5$ selections. Spacing = $1/5 = 0.2$. Start $r = 0.05$.

Pointers:

$0.05, 0.25, 0.45, 0.65, 0.85$

Using the previous cumulative intervals:

Pointer	Selected
0.05	A
0.25	B

0.45	C
0.65	D
0.85	D

Output parent pool:

[A, B, C, D, D]

SUS MCQ recognition

MCQ phrase	Answer
closest to probability distribution	SUS
multiple equally spaced pointers	SUS
multi-arm roulette spun once	SUS
repeated independent spins	roulette, not SUS

5.4 Tournament selection

Theory

Tournament selection samples k individuals and selects the best in that sample.

Step-by-step procedure

1. Choose tournament size k .
2. Randomly sample k individuals.
3. Compare fitness values.
4. Select the best as parent.
5. Repeat until enough parents selected.

Worked example

Fitness:

A=10, B=30, C=15, D=40, E=5

Tournament sample:

[B, C, E]

Compare fitness:

B=30, C=15, E=5

Winner: B.

Tournament traps

Trap	Correction
tournament respects exact probability distribution	no, it is based on sampled competition
tournament requires FPS probabilities	no

tournament always selects global best	only if best is sampled
tournament is deterministic overall	individual tournament can be deterministic, sampling is random

5.5 Rank selection

Theory

Rank selection sorts individuals by fitness and assigns probabilities based on rank, not raw fitness. It reduces the domination problem when one individual has huge raw fitness.

Worked example

Raw fitness:

A=10, B=1000, C=20, D=30

FPS would make B dominate. Rank selection ranks:

B rank 1, D rank 2, C rank 3, A rank 4

Probabilities are assigned from ranks, not raw values.

MCQ traps

Trap	Correction
rank selection uses raw fitness directly	false
rank selection removes selection pressure	false, it moderates it
rank selection is tournament	false

5.6 Sigma scaling and windowing

These are FPS scaling methods.

Sigma scaling

Fitness values are adjusted using the mean and standard deviation to reduce domination by very high fitness values.

Recognition: if an MCQ mentions mean, standard deviation, or sigma scaling as a parent selection adjustment, it is a valid FPS variant.

Windowing

Fitness values are adjusted by subtracting a baseline, often the worst fitness in a recent window.

Recognition: valid FPS variant.

Invalid distractors

Distractor	Why wrong
alpha scaling	often not a valid course-listed FPS model
complicated roulette	not a standard parent selection mechanism
data validation	not selection

5.7 Parent selection practice with solutions

Exercise PS1

Fitness values are A=5, B=15, C=30. Compute FPS probabilities.

Solution:

Total = 50. A=0.10, B=0.30, C=0.60.

Exercise PS2

Using probabilities A=0.10, B=0.30, C=0.60, which individual is selected by $r=0.35$?

Solution:

Cumulative: A 0.00-0.10, B 0.10-0.40, C 0.40-1.00. $r=0.35$ selects B.

Exercise PS3

For SUS with N=4 and start $r=0.10$, list the pointers.

Solution:

Spacing = 0.25. Pointers = 0.10, 0.35, 0.60, 0.85.

Exercise PS4

Tournament sample has individuals A=12, C=40, D=25. Who wins?

Solution: C.

Exercise PS5

Which selection is most associated with "closest to the probability distribution"?

Solution: SUS.

Part III - Recombination and crossover: all manual procedures

6. Recombination overview

Recombination/crossover answers: **how do parents combine to create offspring?** In GA it is usually a major operator. In ES it is secondary and takes different forms.

Representation	Common crossover/recombination
Binary	one-point, multi-point, uniform
Integer	one-point, multi-point, uniform, arithmetic with rounding
Real	arithmetic/blend, one-point, multi-point, uniform
Permutation	PMX, OX/OCX, CX, ECX
ES real vector	local/global, discrete/intermediate

6.1 One-point crossover

Theory

One cut point is selected. Child 1 takes the first part from Parent 1 and the second part from Parent 2. Child 2 takes the opposite.

Step procedure

1. Write parents in aligned positions.
2. Mark the cut point.
3. Child 1 = P1 prefix + P2 suffix.
4. Child 2 = P2 prefix + P1 suffix.
5. Validate representation.

Worked example: binary

```
P1 = 1 1 0 1 | 0 0 1 0
P2 = 0 0 1 0 | 1 1 0 1
```

Output:

```
C1 = 1 1 0 1 1 1 0 1
C2 = 0 0 1 0 0 0 1 0
```

Worked example: integer

```
P1 = [4, 7, 2 | 9, 5]
P2 = [1, 3, 8 | 6, 0]
```

Output:

```
C1 = [4, 7, 2, 6, 0]
C2 = [1, 3, 8, 9, 5]
```

Trap for permutations

If parents are permutations, raw one-point crossover may create duplicates.

Example:

```
P1 = [1,2,3 | 4,5]
P2 = [5,4,3 | 2,1]
C1 = [1,2,3,2,1]  invalid duplicate 1 and 2, missing 4 and 5
```

Therefore for permutations, use PMX/OX/CX/ECX or repair.

6.2 Multi-point crossover

Theory

Multiple cut points are selected. Segments alternate between parents.

Step procedure

1. Mark all cut points.
2. Split parents into segments.
3. Alternate segments for Child 1.

- Use opposite segments for Child 2.
- Validate.

Worked example

Parents:

```
P1 = [A B | C D E | F G]
P2 = [a b | c d e | f g]
```

Cuts after positions 2 and 5.

Output:

```
C1 = [A B c d e F G]
C2 = [a b C D E f g]
```

MCQ traps

Trap	Correction
multi-point always better	no, more disruption can harm
valid for raw permutations	not without repair
same as uniform crossover	no, uniform chooses gene-by-gene

6.3 Uniform crossover

Theory

For each gene, choose the value from Parent 1 or Parent 2 according to a mask or probability, often 0.5.

Step procedure

- Align parents.
- Read the mask.
- If mask says P1, copy gene from P1.
- If mask says P2, copy gene from P2.
- Build child.

Worked example

Parents:

```
P1 = [1,1,0,0,1,0]
P2 = [0,0,1,1,0,1]
Mask = [P1,P2,P2,P1,P1,P2]
```

Steps:

Position	Mask	Child gene
1	P1	1
2	P2	0
3	P2	1
4	P1	0

5	P1	1
6	P2	1

Output:

$C = [1, 0, 1, 0, 1, 1]$

Uniform crossover vs uniform mutation

Uniform crossover	Uniform mutation
uses two parents	uses one chromosome
chooses each gene from P1/P2	replaces a gene with random value
recombination	mutation

6.4 Arithmetic recombination

Theory

Arithmetic recombination creates numeric offspring by weighted averages.

Core formula:

$C1 = \alpha * P1 + (1 - \alpha) * P2$
 $C2 = (1 - \alpha) * P1 + \alpha * P2$

When valid

Representation	Validity
Real	valid directly
Integer	valid with rounding/repair if course allows
Binary	not standard directly
Permutation	invalid

Worked example: real vectors

Parents:

$P1 = [10, 20, 30]$
 $P2 = [20, 10, 40]$
 $\alpha = 0.3$

Child 1:

$C1 = 0.3 * P1 + 0.7 * P2$

Gene	Calculation	Result
1	$0.310 + 0.720$	17
2	$0.320 + 0.710$	13

3	$0.330 + 0.740$	37

$C1 = [17, 13, 37]$

Child 2:

$C2 = 0.7 * P1 + 0.3 * P2 = [13, 17, 33]$

Simple, singular, and total arithmetic recombination

Type	Meaning
Simple arithmetic	choose cut, blend the part after/before cut
Singular arithmetic	blend one selected gene
Total/whole arithmetic	blend all genes

Worked example: singular arithmetic

Parents:

$P1 = [10, 20, 30]$

$P2 = [20, 10, 40]$

$\alpha = 0.5$

selected gene = 2

Only gene 2 is blended.

Gene 2 child = $0.520 + 0.510 = 15$.

$C1 = [10, 15, 30]$

Other genes remain from the chosen base parent depending on convention.

Arithmetic traps

Trap	Correction
use arithmetic on permutations	invalid
forget alpha complement	use alpha and 1-alpha
integer decimal offspring	round/repair/check feasibility
confuse mutation with recombination	arithmetic recombination uses two parents

6.5 ES recombination: local/global and discrete/intermediate

Theory

ES recombination combines real-valued parent vectors. It is secondary to mutation.

Type	How it works
Local	uses two parents
Global	uses multiple parents from population

Discrete	each gene copied from one selected parent
Intermediate	each gene averaged/blended

Worked example: discrete ES recombination

P1 = [2, 4, 6]
P2 = [8, 1, 5]
Mask = [P1, P2, P1]

Output:

C = [2, 1, 6]

Worked example: intermediate ES recombination

P1 = [2, 4, 6]
P2 = [8, 1, 5]
C = [(2+8)/2, (4+1)/2, (6+5)/2]
C = [5, 2.5, 5.5]

ES recombination traps

Trap	Correction
PMX/OX/CX/ECX in ES	those are GA permutation operators
ES recombination is primary	mutation is primary
ES recombination selects survivors	selection is separate

6.6 Standard crossover practice

Exercise C1

P1 = [1,0,1,0,1,0], P2 = [0,0,0,1,1,1], cut after position 3. Compute one-point children.

Solution:

C1 = [1,0,1,1,1,1]. C2 = [0,0,0,0,1,0].

Exercise C2

P1 = [1,2,3,4,5,6], P2=[6,5,4,3,2,1], cuts after positions 2 and 4. Alternate segments starting with P1. Compute C1.

Solution:

Segments: P1 [1,2] | [3,4] | [5,6], P2 [6,5] | [4,3] | [2,1]. C1 = [1,2,4,3,5,6].

Exercise C3

P1=[2,4,6], P2=[8,2,10], alpha=0.25. Compute total arithmetic C1.

Solution:

C1=0.25P1+0.75P2=[6.5,2.5,9].

Exercise C4

Can arithmetic recombination be directly applied to a permutation route [1,2,3,4]? Why?

Solution:

No. Arithmetic averages destroy permutation identity and uniqueness.

Part IV - Permutation crossover workshop: PMX, OX/OCX, CX, ECX

7. Permutation crossover overview

Permutation crossover must produce valid permutations. The output must contain every allele exactly once.

Operator	Preserves	Main trigger	Core procedure
PMX	mapped segment and repaired positions	partially mapped, positions i/j	copy segment, map conflicts, repair
OX/OCX	relative order	order crossover, order of events	copy segment, fill from other parent in order
CX	cycles of absolute positions	cycle crossover, absolute position	build cycles, alternate cycles
ECX	edges/adjacency	edge, common relationships	build edge table, choose adjacent alleles

7.1 PMX - Partially Mapped Crossover

Theory

PMX is used for permutation chromosomes. It copies a segment from one parent and uses a mapping between the two parent segments to repair conflicts when filling the rest from the other parent.

Input and output

Input	Output
two permutation parents + two cut positions	one or two valid permutation children

PMX step procedure for Child 1

1. Write both parents with position numbers.
2. Mark the cut interval [i..j].
3. Copy P1[i..j] into Child 1.
4. Build mappings between P1[i..j] and P2[i..j].
5. For every position outside the segment, try to copy the value from P2.
6. If the value is already inside Child 1, follow the mapping until a value not in the segment appears.
7. Fill the position with the repaired value.
8. Validate: no duplicates and no missing values.

PMX step procedure for Child 2

Repeat symmetrically:

1. Copy P2[i..j].
2. Fill outside positions from P1.

3. Repair conflicts using the same mapping.

Fully solved PMX example from the MCQ style

Parents:

P1 = [6, 3, 7, 8, 5, 1, 2, 4, 9]
P2 = [7, 2, 8, 5, 6, 9, 4, 1, 3]
Cuts = positions 3 and 6 inclusive

Position table:

Position	1	2	3	4	5	6	7	8	9
P1	6	3	7	8	5	1	2	4	9
P2	7	2	8	5	6	9	4	1	3

Copy P1 segment positions 3-6 into Child 1:

C1 = [_, _, 7, 8, 5, 1, _, _, _]

Mapping from segment:

7 <-> 8
8 <-> 5
5 <-> 6
1 <-> 9

Fill outside positions from P2:

Position	P2 value	Conflict?	Mapping repair	C1 value
1	7	yes	7 -> 8 -> 5 -> 6	6
2	2	no	none	2
7	4	no	none	4
8	1	yes	1 -> 9	9
9	3	no	none	3

Child 1:

C1 = [6, 2, 7, 8, 5, 1, 4, 9, 3]

Now Child 2: copy P2 segment positions 3-6:

C2 = [_, _, 8, 5, 6, 9, _, _, _]

Fill outside positions from P1:

Position	P1 value	Conflict?	Mapping repair	C2 value
1	6	yes	6 -> 5 -> 8 -> 7	7
2	3	no	none	3
7	2	no	none	2
8	4	no	none	4

9	9	yes	9 -> 1	1
---	---	-----	--------	---

Child 2:

C2 = [7, 3, 8, 5, 6, 9, 2, 4, 1]

How to eliminate fake PMX options

For every option, check in this order:

1. Is it a valid permutation? If duplicate or missing, eliminate.
2. Does it preserve the copied segment exactly?
3. Are outside positions filled according to opposite parent plus mapping repair?
4. Does it correspond to Child 1 or Child 2? If neither, eliminate.

PMX traps

Trap	Correction
simply copy tail like one-point crossover	PMX maps and repairs conflicts
ignore duplicate values	invalid permutation
copy wrong segment	cut positions must be respected
confuse PMX with OX	OX preserves order, PMX uses mapping

7.2 OX / OCX - Order Crossover

Theory

OX/OCX is used for permutations when relative order matters. It copies a segment from one parent and fills the remaining positions using the order of genes in the other parent, skipping already-used genes.

Core idea

PMX repairs by mapping. OX repairs by preserving order.

Step procedure for Child 1

1. Choose cut interval [i..j].
2. Copy P1[i..j] into Child 1.
3. Starting after cut j in P2, read P2 cyclically.
4. Skip values already in Child 1.
5. Fill empty positions of Child 1 starting after j and wrapping around.
6. Validate permutation.

Fully solved OX example

Parents:

P1 = [1,2,3,4,5,6,7,8,9]
P2 = [9,3,7,8,2,6,5,1,4]
Cuts = 3 and 6

Copy P1 segment positions 3-6:

C1 = [_, _, 3, 4, 5, 6, _, _, _]

Read P2 after position 6 cyclically:

P2 order after cut = [5,1,4,9,3,7,8,2,6]

Remove already-used values 3,4,5,6:

remaining order = [1,9,7,8,2]

Fill empty positions starting after cut: positions 7,8,9,1,2.

Fill position	Value
7	1
8	9
9	7
1	8
2	2

Output:

C1 = [8,2,3,4,5,6,1,9,7]

Child 2

For Child 2, copy segment from P2 and fill using P1 order.

Copy P2 positions 3-6:

C2 = [_, _, 7, 8, 2, 6, _, _, _]

Read P1 after position 6 cyclically:

[7,8,9,1,2,3,4,5,6]

Remove values already used 7,8,2,6:

[9,1,3,4,5]

Fill positions 7,8,9,1,2:

C2 = [4,5,7,8,2,6,9,1,3]

OX/OCX MCQ triggers

Phrase	Meaning
order crossover	OX/OCX
preserves order of events	OX/OCX
relative order	OX/OCX
fill from other parent in order	OX/OCX

OX traps

Trap	Correction
------	------------

use PMX mapping	OX uses order filling, not mapping
preserve exact absolute positions	that is CX-like, not OX
duplicate genes	invalid
fill from the start instead of after cut	convention error; course may specify, default is after cut/wrap

7.3 CX - Cycle Crossover

Theory

CX is used for permutations. It preserves absolute positions by cycles. A cycle is formed by following values between parents until returning to the starting position.

Core idea

If a position belongs to a cycle taken from P1, Child 1 copies P1 at all those positions. Other cycles are copied from P2. Child 2 takes the opposite.

Step procedure

1. Write both parents with positions.
2. Start at the first unused position.
3. At that position, look at the value in P2.
4. Find that value's position in P1.
5. Move to that position.
6. Repeat until you return to the starting position. That is one cycle.
7. Start a new cycle at the next unused position.
8. Child 1 takes odd cycles from P1 and even cycles from P2.
9. Child 2 takes odd cycles from P2 and even cycles from P1.
10. Validate the children.

Fully solved CX example

Parents:

P1 = [1,2,3,4,5,6,7,8]
P2 = [3,7,5,1,6,8,2,4]

Position table:

Position	1	2	3	4	5	6	7	8
P1	1	2	3	4	5	6	7	8
P2	3	7	5	1	6	8	2	4

Cycle 1:

Step	Current position	P2 value	Position of that value in P1
1	1	3	3
2	3	5	5
3	5	6	6

4	6	8	8
5	8	4	4
6	4	1	1 closes cycle

Cycle 1 positions:

{1, 3, 5, 6, 8, 4}

Unused positions: 2 and 7.

Cycle 2:

Step	Current position	P2 value	Position in P1
1	2	7	7
2	7	2	2 closes cycle

Cycle 2 positions:

{2, 7}

Build children.

Child 1 takes Cycle 1 from P1 and Cycle 2 from P2:

Position	Cycle	Source	Value
1	C1	P1	1
2	C2	P2	7
3	C1	P1	3
4	C1	P1	4
5	C1	P1	5
6	C1	P1	6
7	C2	P2	2
8	C1	P1	8

Child 1 = [1, 7, 3, 4, 5, 6, 2, 8]

Child 2 takes Cycle 1 from P2 and Cycle 2 from P1:

Child 2 = [3, 2, 5, 1, 6, 8, 7, 4]

CX option elimination

Check each option:

1. Valid permutation?
2. Does each position come entirely from P1 or P2 according to full cycles?
3. Are cycle positions mixed incorrectly? If yes, eliminate.
4. Does it match Child 1 or Child 2?

CX traps

Trap	Correction
alternate individual positions	wrong; alternate cycles, not positions
copy a segment	segment-based is PMX/OX, not CX
preserve relative order	that is OX
corrupt options	compute cycles and trust method

7.4 ECX - Edge Recombination Crossover

Theory

ECX preserves adjacency/edges. It is important for TSP-like problems where relationships between neighboring cities matter.

Core idea

Build an edge table from both parents. Then create a child by choosing next alleles that preserve common or promising edges.

Step procedure

1. Treat each parent as a cycle if the problem is TSP-like.
2. For each allele, list its left and right neighbors in both parents.
3. Combine neighbors into an edge table.
4. Start from a chosen allele.
5. Remove the current allele from all edge lists.
6. Choose the next allele from the current allele's edge list. Prefer common edges or smallest edge list.
7. If no neighbors remain, choose any unused allele.
8. Continue until all alleles are used.
9. Validate permutation.

Fully solved ECX example

Parents:

P1 = [1,2,3,4,5,6]
P2 = [1,3,5,2,4,6]

Build edge lists. For circular routes:

Allele	Neighbors from P1	Neighbors from P2	Combined edge list
1	6,2	6,3	{2,3,6}
2	1,3	5,4	{1,3,5,4}
3	2,4	1,5	{2,4,1,5}
4	3,5	2,6	{3,5,2,6}
5	4,6	3,2	{4,6,3,2}
6	5,1	4,1	{5,1,4}

Start at 1.

1. Current = 1. Candidates = {2,3,6}. Choose 6 because it has fewer remaining edges than 2 or 3.

2. Current = 6. Candidates = {5,4}. Choose 5.
3. Current = 5. Candidates = {4,3,2}. Choose 4.
4. Current = 4. Candidates = {3,2}. Choose 2.
5. Current = 2. Candidates = {3}. Choose 3.
6. All alleles used.

One possible child:

[1, 6, 5, 4, 2, 3]

Different ECX tie-breaks may yield different valid children. In MCQs, use the specified tie-break if given; otherwise check edge preservation and validity.

ECX triggers and traps

Trigger	Answer
common edges	ECX
adjacency relations	ECX
edge table	ECX
relative order	OX, not ECX
absolute positions	CX, not ECX
mapping conflicts	PMX, not ECX

7.5 PMX/OX/CX/ECX comparison page

Question	PMX	OX/OCX	CX	ECX
Uses cut segment?	yes	yes	no segment; cycles	no segment; edges
Uses mapping?	yes	no	no	no
Preserves relative order?	partly	yes	no	no
Preserves absolute positions?	partly	no	yes by cycles	no
Preserves edges?	not primarily	not primarily	no	yes
Main MCQ word	mapped	order	cycle	edge/common relationship

7.6 Permutation crossover practice with solutions

Exercise PC1 - PMX

P1=[1,2,3,4,5,6,7,8], P2=[3,7,5,1,6,8,2,4], cuts 3-5. Compute Child 1 by copying P1 segment.

Solution:

Copy P1 segment positions 3-5: [3,4,5]. Mapping: 3<->5, 4<->1, 5<->6. Start C1=[,,3,4,5,,,]. Fill from P2 outside positions: pos1=3 conflict -> 5 conflict -> 6 OK; pos2=7 OK; pos6=8 OK; pos7=2 OK; pos8=4 conflict -> 1 OK. C1=[6,7,3,4,5,8,2,1].

Exercise PC2 - OX

P1=[1,2,3,4,5,6,7,8], P2=[3,7,5,1,6,8,2,4], cuts 3-5. Compute Child 1 by copying P1 segment.

Solution:

Copy [3,4,5] to positions 3-5. Read P2 after cut position 5: [8,2,4,3,7,5,1,6]. Skip 3,4,5 -> [8,2,7,1,6]. Fill positions 6,7,8,1,2.
C1=[1,6,3,4,5,8,2,7].

Exercise PC3 - CX

Use P1=[1,2,3,4,5,6,7,8], P2=[3,7,5,1,6,8,2,4]. What are the cycle positions?

Solution:

Cycle 1={1,3,5,6,8,4}. Cycle 2={2,7}.

Exercise PC4 - Identify operator

An operator builds an edge table and tries to preserve shared neighbors. Which operator is it?

Solution: ECX.

Exercise PC5 - Validity check

Is [1,2,3,4,5,5,7,8] a valid permutation child for parents containing 1-8?

Solution: No. It duplicates 5 and misses 6.

Part V - Mutation and variation: all manual procedures

8. Mutation overview

Mutation changes one chromosome/candidate. It does not combine two parents. In GA it is usually a secondary operator; in ES it is central.

Representation	Mutation types
Binary	bit flip / negation
Integer	creep/fluaj, random reset, bounded reset
Real	uniform, non-uniform, Gaussian/local
Permutation	swap, insertion, inversion, scramble/amestec
ES	Gaussian/non-uniform, sigma self-adaptation

8.1 Mutation identification decision tree

Given before/after chromosome:

Are values 0/1 and bits flip?
-> bit flip

Are values integers and one/few change slightly by +/- small step?
-> creep/fluaj

Is one gene replaced by an unrelated valid value?
-> random reset

Are values real and changed by numeric perturbation?
-> Gaussian/uniform/non-uniform mutation depending wording

Is it a permutation?
Exactly two positions differ and are exchanged?
-> swap

One element moved and a block shifted?
-> insertion

A contiguous segment is reversed?
-> inversion

Same values inside interval but shuffled randomly?
-> scramble/amestec

8.2 Bit flip mutation

Procedure

1. Identify selected bit positions.
2. Change 0 to 1 and 1 to 0.
3. Leave all other bits unchanged.

Example

```
X = [1,0,1,1,0,0]
positions = 2,5
X' = [1,1,1,1,1,0]
```

Traps

Trap	Correction
bit flip on real values	invalid
bit flip on permutation	invalid
bit flip changes value to any integer	no, only 0<->1

8.3 Creep/fluaj mutation

Theory

Creep/fluaj mutation applies a small local integer change. It is common for integer representations.

Procedure

1. Identify mutated gene.
2. Add or subtract a small integer step.
3. Check bounds.
4. Keep if valid; repair if out of bounds.

Example

```
X = [5, 8, 2, 6]
position 2, step -1
```

$X' = [5, 7, 2, 6]$

MCQ recognition

Before/after	Mutation
8 -> 7	creep/fluj
2 -> 3	creep/fluj
2 -> 9	random reset, not creep unless large steps allowed

8.4 Random reset mutation

Procedure

1. Choose a gene.
2. Replace it with a new valid value from the domain.
3. Validate bounds/domain.

Example:

```
X = [5, 8, 2, 6]
position 3 reset to 9
X' = [5, 8, 9, 6]
```

8.5 Uniform mutation for real values

Theory

Uniform mutation replaces a real value with a new value drawn uniformly from its allowed interval.

Procedure

1. Choose gene i .
2. Determine bounds $[L_i, U_i]$.
3. Draw new value uniformly from $[L_i, U_i]$.
4. Replace x_i with that value.

Example

```
X = [0.2, 0.8, 0.4]
mutation gene 2, bounds [0,1], random value 0.31
X' = [0.2, 0.31, 0.4]
```

8.6 Non-uniform mutation for real values

Theory

Non-uniform mutation is generation-dependent. Early generations allow larger moves; later generations make smaller moves.

Procedure concept

1. Choose gene.
2. Choose direction toward lower or upper bound.
3. Compute a generation-dependent delta.

4. Add/subtract delta.
5. Repair if needed.

Example concept

```
x_i = 5.0, upper = 10.0  
large early delta = 2.0 -> x_i' = 7.0  
small late delta = 0.2 -> x_i' = 5.2
```

MCQ recognition

Non-uniform mutation is real-valued and often linked to generation number or changing distribution. It is not scramble, inversion, or creep.

8.7 Gaussian mutation

Theory

Gaussian mutation adds normally distributed noise.

Formula:

```
x_i' = x_i + sigma * N(0,1)
```

Example

```
X = [2.0, 5.0]  
sigma = 0.5  
noise = [0.3, -0.7]  
X' = [2.15, 4.65]
```

GA vs ES distinction

In GA	In ES
valid for real-valued GA mutation	central/main mutation mechanism
applied with mutation probability	often probability 1
sigma usually parameter	sigma may self-adapt

8.8 Swap mutation

Theory

Swap mutation exchanges two positions in a permutation.

Procedure

1. Choose two positions i and j.
2. Exchange the values.
3. Check permutation validity.

Example

```
X = [1,2,3,4,5]  
swap positions 2 and 5  
X' = [1,5,3,4,2]
```

Detection

Exactly two positions differ, and the values are exchanged.

8.9 Insertion mutation

Theory

Insertion removes one element and inserts it elsewhere, shifting the intervening block.

Procedure

1. Choose source position i .
2. Remove value at i .
3. Choose target insertion position j .
4. Insert value at j .
5. Shift other elements accordingly.
6. Validate permutation.

Example

```
X = [1,2,3,4,5,6]
remove position 2 value 2
insert after position 5 in original idea
X' = [1,3,4,5,2,6]
```

Detection

One value moved, and the values between source and destination shift by one.

8.10 Inversion mutation

Theory

Inversion reverses a contiguous segment.

Procedure

1. Choose positions i and j .
2. Reverse the subsequence $X[i..j]$.
3. Keep outside positions unchanged.
4. Validate permutation.

Example

```
X = [1,2,3,4,5,6,7]
invert positions 3..6
segment [3,4,5,6] -> [6,5,4,3]
X' = [1,2,6,5,4,3,7]
```

Detection

A contiguous interval is exactly reversed.

8.11 Scramble / amestec mutation

Theory

Scramble mutation shuffles values inside a selected interval. The same values remain inside the interval, but in a new order.

Procedure

1. Choose positions i and j .
2. Extract segment $X[i..j]$.
3. Randomly permute/shuffle the segment.
4. Put shuffled segment back.
5. Outside interval unchanged.
6. Validate permutation.

Example

```
X = [1,2,3,4,5,6,7]
scramble positions 3..6
segment [3,4,5,6] -> [5,3,6,4]
X' = [1,2,5,3,6,4,7]
```

Detect cut positions from before/after

1. Compare from left; first changed position = left cut.
2. Compare from right; first changed position from right = right cut.
3. Check that the interval contains the same values.
4. If yes, scramble.

Solved detection example

```
Original = [7,6,12,14,3,10,8,15,11,5,4,1,13,2,9]
Mutated  = [7,14,13,12,1,15,2,8,6,3,11,5,10,4,9]
```

First changed from left: position 2. First changed from right: position 14. Values in positions 2-14 are the same set but shuffled. Therefore: scramble/amestec, positions 2 and 14.

8.12 Mutation comparison table

Mutation	Representation	Before/after signature
Bit flip	binary	0<->1
Creep/fluaj	integer	small numerical change
Random reset	integer/discrete	one value replaced by another valid value
Uniform mutation	real	value replaced from interval
Non-uniform mutation	real	generation-dependent real change
Gaussian mutation	real/ES	add normal noise
Swap	permutation	exactly two positions exchanged
Insertion	permutation	one value moved; block shifts
Inversion	permutation	interval reversed
Scramble	permutation	interval shuffled

--	--	--

8.13 Mutation practice with solutions

Exercise M1

Original [1,2,3,4,5], mutated [1,5,3,4,2]. Mutation?

Solution: swap positions 2 and 5.

Exercise M2

Original [1,2,3,4,5,6], mutated [1,2,6,5,4,3]. Mutation?

Solution: inversion positions 3-6.

Exercise M3

Original [1,2,3,4,5,6], mutated [1,3,4,2,5,6]. Mutation?

Solution: insertion; value 2 moved from position 2 to position 4, shifting 3 and 4 left.

Exercise M4

Original [1,2,3,4,5,6], mutated [1,4,2,5,3,6]. Mutation?

Solution: scramble positions 2-5 because values {2,3,4,5} are shuffled.

Exercise M5

Original [5,8,2,6], mutated [5,7,2,6]. Mutation?

Solution: creep/fluaj at position 2 with step -1.

Exercise M6

Original [0,1,1,0], mutated [0,0,1,1]. Mutation?

Solution: bit flip at positions 2 and 4.

Exercise M7

Original [2.0,5.0], sigma=0.2, noise=[1.0,-0.5]. Compute Gaussian mutation.

Solution: [2.2,4.9].

Part VI - Survivor selection, replacement, elitism, and ES selection

9. Parent selection vs survivor selection

Question	Parent selection	Survivor selection
Main question	who reproduces?	who remains?
Timing	after evaluation, before recombination	after offspring are created/evaluated
Input	current evaluated population	current population, offspring, or both

Output	parent/mating pool	next generation
Common methods	FPS, roulette, SUS, tournament, rank	generational, steady-state, elitism, age, quality, mu/lambda

9.1 Generational replacement

Theory

The entire parent generation is replaced by offspring.

Procedure

1. Generate offspring population of same size as parent population.
2. Replace all parents with offspring.
3. Optionally preserve elite if elitism is added.

Example

Current population:

P = [A,B,C,D]

Offspring:

O = [E,F,G,H]

Generational replacement:

Next = [E,F,G,H]

If elitism keeps best parent B and only 3 offspring fit:

Next = [B,E,F,G]

9.2 Steady-state replacement

Theory

Only a few individuals are replaced at each iteration.

Procedure

1. Generate one or a few offspring.
2. Choose which current individuals to replace, often worst or oldest.
3. Insert offspring.

Example

Current:

A=50, B=40, C=30, D=20

New offspring:

E=45

Replace worst D=20:

Next = [A=50, B=40, C=30, E=45]

9.3 GENITOR

GENITOR is a steady-state style replacement often replacing one individual, frequently the worst, with one offspring.

Trap

GENITOR can converge prematurely because it may quickly remove diversity.

9.4 Elitism

Theory

Elitism copies the best individual(s) unchanged into the next generation.

Procedure

1. Identify best k individuals in current population.
2. Copy them into next generation unchanged.
3. Fill remaining slots with offspring or selected survivors.

Example

Current:

A=100, B=70, C=60, D=50

Offspring:

E=90, F=80, G=40, H=30

Generational replacement without elitism:

Next = [E,F,G,H]

Best A=100 is lost.

With elitism k=1:

Next = [A,E,F,G] or [A,E,F,H] depending fill rule

Elitism traps

Trap	Correction
elitism selects parents	no, it preserves survivors
elitism means all parents survive	no, only best k
elitism increases diversity	usually reduces diversity risk
elitism guarantees global optimum	no, only preserves current best

9.5 Superelitism

Superelitism preserves more elites or gives stronger protection to top individuals. It can speed convergence but increase loss of diversity.

9.6 Age-based survivor selection

Theory

Individuals are removed based on age, not necessarily quality.

Example

Current:

Individual	Fitness	Age
A	100	10
B	40	2
C	30	1
D	20	1

If age-based removes oldest, A may be removed even though it is best.

Trap

Age-based selection does not guarantee improved average quality.

9.7 Quality-based survivor selection

Select the best fitness individuals from a candidate pool.

Example

Current:

A=50, B=40, C=35, D=30, E=25

Offspring:

F=45, G=55, H=20, I=60, J=15, K=48

Best 5 from current+offspring:

I=60, G=55, A=50, K=48, F=45

9.8 ES survivor selection: (μ, λ) and ($\mu + \lambda$)

(μ, λ)

Select μ survivors from λ offspring only. Parents are discarded.

($\mu + \lambda$)

Select μ survivors from parents + offspring. Elitist because good parents can survive.

Worked example

Parents:

P = [100, 70, 60]

Offspring:

O = [90, 80, 50, 40, 30]

(mu+lambda), mu=3:

Pool = [100, 90, 80, 70, 60, 50, 40, 30]

Next = [100, 90, 80]

(mu,lambda), mu=3:

Pool = [90, 80, 50, 40, 30]

Next = [90, 80, 50]

ES selection traps

Trap	Correction
(mu,lambda) is elitist	false
(mu+lambda) uses offspring only	false
roulette selection in ES survivor selection	false in core ES
best parent always survives in (mu,lambda)	false

9.9 Survivor selection practice with solutions

Exercise S1

Current: A=10, B=50, C=30. Offspring: D=40, E=20, F=60. Choose best 3 from current+offspring.

Solution: F=60, B=50, D=40.

Exercise S2

Same data, generational replacement with offspring only.

Solution: D, E, F.

Exercise S3

Current: A fitness 100 age 9, B fitness 50 age 1. If age-based removes oldest, who is removed?

Solution: A, despite higher fitness.

Exercise S4

Parents [70,60], offspring [65,55,50]. Compute (mu+lambda) and (mu,lambda) with mu=2.

Solution: (mu+lambda) -> [70,65]. (mu,lambda) -> [65,55].

Exercise S5

Which model preserves the best current individual automatically if it remains best: (mu,lambda) or (mu+lambda)?

Solution: $(\mu + \lambda)$.

Part VII - Evolution Strategies full application reference

10. ES representation, mutation, recombination, selection

10.1 ES representation

An ES individual is often represented as:

(x, σ)

or with one step size per component:

$(x, \sigma_1, \sigma_2, \dots, \sigma_n)$

Part	Meaning
x	object variables / actual candidate solution
sigma	mutation step size / strategy parameter
sigma_i	mutation step size for variable i

Example

Individual = $([4.0, 2.0, 7.0], \sigma=0.3)$

This means the solution is $[4.0, 2.0, 7.0]$, and mutation magnitude is controlled by $\sigma=0.3$.

10.2 ES mutation with sigma

Formula

$x_i' = x_i + \sigma * z_i$

where z_i is Gaussian noise.

Example

$x = [4.0, 2.0, 7.0]$
 $\sigma = 0.3$
 $z = [0.5, -0.2, 0.8]$

Output:

$x' = [4.15, 1.94, 7.24]$

10.3 ES self-adaptation

Theory

Self-adaptation means strategy parameters evolve together with object variables.

Common simplified formula:

```
sigma' = sigma * exp(tau * N(0,1))
x_i' = x_i + sigma' * N_i(0,1)
```

Manual steps

1. Mutate sigma first.
2. Use new sigma' to mutate x.
3. Evaluate candidate.
4. Select survivors.

Example

```
sigma = 1.0
tau*N(0,1) = 0.3
sigma' = 1.0*exp(0.3) = 1.3499
x = [2.0, 5.0]
z = [0.4, -0.6]
x' = [2.0 + 1.3499*0.4, 5.0 + 1.3499*(-0.6)]
x' = [2.540, 4.190]
```

10.4 ES-2M / two-member strategy

Recognition

Phrase	Meaning
one candidate solution maintained	ES-2M
Gaussian mutation for every component	ES-2M/ES
continuous space	ES-2M
stochastic calculation	ES-2M
large population	false for ES-2M

Procedure

1. Keep current candidate x.
2. Create $y = x + \text{Gaussian mutation}$.
3. Evaluate $f(y)$ and $f(x)$.
4. If y better, accept y.
5. Otherwise keep x.

Example

Maximize:

```
f(x) = -(x-3)^2
x = 1
mutation z = 0.7
y = 1.7
```

Evaluate:

```
f(1) = -4
f(1.7) = -1.69
```

Since $-1.69 > -4$, accept $y = 1.7$.

10.5 ES vs GA exam traps

MCQ statement	GA	ES	Correct exam move
mutation low probability	yes	no	GA
mutation probability 1	no	yes	ES
binary canonical representation	yes	no	GA
real vector with sigma	no	yes	ES
FPS selection	yes	not defining ES	GA
PMX/OX/CX/ECX	GA permutation	no	GA
local/global recombination	less typical wording	yes	ES
(mu,lambda)	no	yes	ES
Gaussian mutation	possible in real GA	central in ES	wording decides

10.6 ES practice with solutions

Exercise ES1

$x=[1.0, 2.0]$, $\sigma=0.5$, $z=[1.0, -2.0]$. Compute x' .

Solution: $x'=[1.5, 1.0]$.

Exercise ES2

Parents $[2, 4, 6]$ and $[8, 1, 5]$. Compute intermediate recombination.

Solution: $[5, 2.5, 5.5]$.

Exercise ES3

In (μ, λ) , can a parent survive directly?

Solution: No. Selection is from offspring only.

Exercise ES4

Which is ES-safe: low mutation probability, FPS selection, mutation probability 1, PMX crossover?

Solution: mutation probability 1.

Exercise ES5

What does σ control?

Solution: mutation step size/magnitude.

Part VIII - Application and MCQ decision frameworks

11. Problem -> representation -> operator map

Problem type	Representation	Fitness idea	Mutation	Recombination	Trap
--------------	----------------	--------------	----------	---------------	------

0-1 knapsack	binary	value with capacity constraint	bit flip	one-point/uniform	infeasible capacity
continuous knapsack	real	value under capacity	non-uniform/ Gaussian	arithmetic	needs repair
TSP	permutation	inverse route cost	inversion/swap	PMX/OX/ECX	duplicate cities invalid
N-Queens	permutation	non-attacking pairs	swap/insertion	OX/PMX	representation constraints
Job-shop scheduling	permutation/order	inverse makespan	insertion/swap	PMX/OX	precedence constraints
transportation/ routing	permutation/order	inverse cost	swap/inversion	CX/PMX	decoding matters
maze/directions	integer	path score	creep/reset	one-point/uniform	direction encoding
portfolio weights	real	return-risk score	Gaussian/non- uniform	arithmetic	weights may need sum-to-one repair

11.1 Decision tree for choosing representation

Does the solution mean yes/no inclusion?

-> binary

Does each gene have a discrete integer/category value?

-> integer

Are variables continuous real quantities?

-> real

Does the solution represent an order/route/schedule where each item appears once?

-> permutation

11.2 Decision tree for choosing operator

What representation is given?

Binary:

mutation -> bit flip

crossover -> one-point/multi-point/uniform

Integer:

mutation -> creep or reset

crossover -> standard or arithmetic with rounding

Real:

mutation -> uniform/non-uniform/Gaussian

crossover -> arithmetic/uniform/standard

Permutation:

mutation -> swap/insertion/inversion/scramble

crossover -> PMX/OX/CX/ECX

ES real vector:

mutation -> Gaussian with probability 1, possibly self-adaptive sigma

recombination -> local/global, discrete/intermediate

11.3 Question-type to solving procedure map

If MCQ asks...	Run this procedure
offspring after PMX	copy segment, map, repair, validate
offspring after OX/OCX	copy segment, fill in other parent's order
offspring after CX	build cycles, alternate cycles
offspring after ECX	build edge table, preserve adjacency
mutation type from before/after	use mutation identification tree
positions used in scramble	first changed from left/right
valid operators for real representation	Gaussian/uniform/non-uniform mutation, arithmetic crossover
valid operators for permutation	PMX/OX/CX/ECX, swap/insertion/inversion/scramble
parent selection probability	compute FPS/cumulative or SUS pointers
survivor selection output	identify pool then choose survivors
ES mutation	$x' = x + \sigma \cdot z$, probability 1
(mu,lambda) vs (mu+lambda)	offspring only vs parents+offspring

11.4 Corrupted-key / exam-risk register

Topic	Theory-safe answer	Exam-risk issue	Safe behavior
CX offspring	compute cycles	keys can be corrupted	trust procedure and note risk
Integer arithmetic crossover	valid with rounding/repair	some keys may say invalid	prefer rounded theory unless exact key repeats
Initial population	before evolution, random/often uniform	feasible-only appears	answer exact wording
Survivor selection	current/offspring/both depending model	answer sets vary	inspect wording
Parent selection	generally stochastic	deterministic variants exist	if "in general", choose nondeterministic
ES representation	course often real vectors	some broad keys include any rep	use ES-specific wording
TSP PMX/ECX	PMX in some course examples, ECX for edges	both appear	use wording: edge/common -> ECX
Hillclimbing	local search, not EA core	some keys odd	use property set
Memetic ECX	ECX is crossover, not local improvement	possible mistranslation	mark risk

Part IX - Final practice bank with solutions

12. Mixed practice: representation and compatibility

Exercise R1

A chromosome encodes whether each item is selected in a knapsack. Which representation? Which mutation?

Solution: binary; bit flip.

Exercise R2

A chromosome encodes a route visiting cities 1 to 10 exactly once. Which representation? Which crossovers?

Solution: permutation; PMX, OX/OCX, CX, ECX.

Exercise R3

A chromosome is [0.23, 0.91, 0.12]. Which mutation types are valid?

Solution: uniform, non-uniform, Gaussian/local mutation.

Exercise R4

A chromosome is [3,7,2,9] where each gene is a discrete quantity. Which mutation is a small local adjustment?

Solution: creep/fluaj.

13. Mixed practice: crossover

Exercise X1

$P1=[1,2,3,4,5]$, $P2=[5,4,3,2,1]$, one-point cut after 2. Compute raw child 1. Is it valid permutation?

Solution: $C1=[1,2,3,2,1]$. Not valid as permutation because duplicates 1 and 2, missing 4 and 5.

Exercise X2

$P1=[1,2,3,4,5,6]$, $P2=[6,5,4,3,2,1]$, uniform mask [P1,P2,P1,P2,P1,P2]. Compute child.

Solution: [1,5,3,3,5,1]. If representation is not permutation, valid. If permutation, invalid due duplicates.

Exercise X3

$P1=[10,20]$, $P2=[30,40]$, $\alpha=0.25$. Compute $C1$.

Solution: $C1=0.25P1+0.75P2=[25,35]$.

Exercise X4

Which operator preserves relative order in permutations?

Solution: OX/OCX.

Exercise X5

Which operator preserves edges/common adjacency?

Solution: ECX.

Exercise X6

Which operator uses cycle construction?

Solution: CX.

Exercise X7

Which operator maps conflicts after copying a segment?

Solution: PMX.

14. Mixed practice: mutation

Exercise Y1

Before [1,2,3,4,5], after [1,4,3,2,5]. Mutation?

Solution: swap positions 2 and 4.

Exercise Y2

Before [1,2,3,4,5], after [1,4,3,2,5]. Could this also be inversion positions 2..4?

Solution: Yes, because segment [2,3,4] reversed becomes [4,3,2]. Some before/after patterns can match more than one operation. If only exactly two positions differ, swap is simplest; if a segment reversal is specified, inversion is valid. Use MCQ wording.

Exercise Y3

Before [1,2,3,4,5,6], after [1,2,5,4,3,6]. Mutation?

Solution: inversion positions 3..5.

Exercise Y4

Before [1,2,3,4,5,6], after [1,4,2,5,3,6]. Mutation?

Solution: scramble positions 2..5.

Exercise Y5

Before [2.0,3.0], sigma=0.1, z=[-2,5]. After?

Solution: [1.8,3.5].

15. Mixed practice: selection

Exercise Z1

Fitness A=20, B=30, C=50. Compute FPS probabilities.

Solution: total=100. A=0.20, B=0.30, C=0.50.

Exercise Z2

Using Exercise Z1, random r=0.62 selects?

Solution: cumulative A 0-0.2, B 0.2-0.5, C 0.5-1.0. Select C.

Exercise Z3

SUS needs 4 selections and start=0.12. List pointers.

Solution: spacing=0.25; pointers 0.12,0.37,0.62,0.87.

Exercise Z4

Tournament sample [A=10, B=40, C=25]. Winner?

Solution: B.

Exercise Z5

Which selection is closest to probability distribution, roulette or SUS?

Solution: SUS.

16. Mixed practice: survivor selection

Exercise V1

Current [A=80,B=50,C=30], offspring [D=70,E=90,F=20]. Best 3 from current+offspring?

Solution: E=90, A=80, D=70.

Exercise V2

Same data, offspring-only generational replacement?

Solution: [D,E,F].

Exercise V3

Same data, elitism k=1 plus two best offspring?

Solution: Preserve A=80; best offspring E=90,D=70. Next [A,E,D] or [E,A,D] order irrelevant.

Exercise V4

ES parents [100,60], offspring [90,50,40], $\mu=2$. Compute (μ, λ) and $(\mu+\lambda)$.

Solution: $(\mu, \lambda)=[90,50]$. $(\mu+\lambda)=[100,90]$.

17. Final operator recognition drill

Prompt	Answer
maps conflicts after copied segment	PMX
fills in order from other parent	OX/OCX
follows cycles between parent positions	CX
builds edge table	ECX
reverses interval	inversion
shuffles interval	scramble
exchanges two positions	swap
one value moved and block shifts	insertion

small integer step	creep/fluaj
real noise added	Gaussian mutation
replaces real value from interval	uniform mutation
generation-dependent real mutation	non-uniform mutation
probability proportional to fitness	FPS/roulette
equally spaced pointers	SUS
random subset best wins	tournament
preserve best unchanged	elitism
offspring-only ES	(mu,lambda)
parents+offspring ES	(mu+lambda)

Part X - Final mastery checklist

Before the exam, you should be able to do the following without looking:

1. Given a representation, list valid mutations and crossovers.
2. Given two binary/integer/real parents, apply one-point, multi-point, uniform, or arithmetic crossover.
3. Given two permutation parents and cuts, apply PMX.
4. Given two permutation parents and cuts, apply OX/OCX.
5. Given two permutation parents, build CX cycles and children.
6. Given edge wording, identify ECX.
7. Given before/after chromosome, identify swap, insertion, inversion, scramble, creep, bit flip, reset, Gaussian.
8. Compute FPS probabilities and cumulative intervals.
9. Compute SUS pointers.
10. Identify tournament selection.
11. Distinguish parent selection from survivor selection.
12. Compute next generation under elitism, generational, steady-state, age, quality.
13. Compute (mu,lambda) and (mu+lambda).
14. Distinguish GA from ES at every component.
15. Recognize corrupted/risky key situations and trust procedure.

Final one-page mental commands

PMX = copy segment, map conflicts, repair.
 OX = copy segment, fill remaining in order from other parent.
 CX = build cycles, alternate cycles.
 ECX = build edge table, preserve adjacency.
 Swap = two positions exchanged.
 Insertion = one element moved, block shifts.
 Inversion = interval reversed.
 Scramble = interval shuffled.
 Creep = small integer step.
 Gaussian = add sigma*noise.
 FPS = probability proportional to fitness.
 SUS = equally spaced pointers.
 Tournament = sample group, best wins.
 Parent selection = who reproduces.

Survivor selection = who remains.
Elitism = best copied unchanged.
(mu,lambda) = offspring only.
(mu+lambda) = parents plus offspring.
GA = crossover-centered canonical model.
ES = mutation-centered, real vector, sigma, probability 1.

Part XI - Deep MCQ option-elimination labs

This part exists because many exam questions do not ask you to explain an operator. They give answer options and ask which outputs are possible. The fastest way to solve these is not to fully recompute every option. It is to eliminate impossible options first, then compute the exact result only among the survivors.

18. PMX option-elimination lab

The PMX elimination ladder

Use this ladder for any PMX MCQ with answer options.

1. **Length check.** If the option has the wrong length, eliminate.
2. **Allele-set check.** If it has a duplicate or missing allele, eliminate.
3. **Segment check.** If it is supposed to be Child 1, the copied P1 segment must appear exactly at the cut positions. If Child 2, the copied P2 segment must appear exactly there.
4. **Outside-source check.** Outside the segment, values must come from the opposite parent unless mapping repair changes them.
5. **Mapping-chain check.** Every conflict must be repaired through the mapping chain.
6. **Symmetry check.** If the question asks for both offspring, look for one child built from P1 segment and one from P2 segment.

PMX worked MCQ with options

Parents:

P1 = [6,3,7,8,5,1,2,4,9]
P2 = [7,2,8,5,6,9,4,1,3]
Cuts = 3..6

Computed Child 1 under inclusive-cut PMX:

C1 = [6,2,7,8,5,1,4,9,3]

Computed Child 2:

C2 = [7,3,8,5,6,9,2,4,1]

Now evaluate options:

Option	Candidate	Decision	Reason
A	[6,3,7,8,5,9,4,1,2]	eliminate	segment positions 3..6 should be [7,8,5,1] for C1 or [8,5,6,9] for C2; this has [7,8,5,9]
B	[7,4,6,8,5,1,2,9,3]	eliminate	segment does not match C1 or C2
C	[8,3,7,5,6,9,4,2,1]	eliminate	segment does not match C1/C2;

			also wrong mapping
D	[6,2,7,8,5,1,4,9,3]	keep	matches C1 exactly
E	[7,2,8,5,6,1,3,4,9]	eliminate	segment partly wrong for C2: position 6 should be 9
F	[6,2,7,8,5,9,4,1,3]	eliminate	position 6 wrong for C1 segment; should be 1

If the official key differs, the likely cause is a convention mismatch or corrupted key. Your procedure is the anchor.

PMX convention warning

Some courses define cut points as positions included in the segment. Others define them as boundaries between genes. Always read the wording. If the question says "positions 3 and 6" in a permutation operator, the common MCQ convention is usually inclusive positions 3 through 6.

19. OX/OCX option-elimination lab

OX elimination ladder

1. Check valid permutation.
2. Check copied segment exactly matches the selected source parent.
3. Remove copied segment values from the other parent.
4. Check that remaining genes appear in the correct relative order.
5. Remember: OX preserves order, not mapping.

OX worked MCQ with options

Parents:

P1 = [1,2,3,4,5,6,7,8,9]
P2 = [9,3,7,8,2,6,5,1,4]
Cuts = 3..6

Child 1 from P1 segment:

C1 = [8,2,3,4,5,6,1,9,7]

Why? Segment [3,4,5,6] is copied. Remaining order from P2 after cut is [1,9,7,8,2]. Fill positions 7,8,9,1,2.

Fake options:

Candidate	Decision	Reason
[9,2,3,4,5,6,7,8,1]	eliminate	remaining values are not in P2 cyclic order
[8,2,3,4,5,6,1,9,7]	keep	exact C1
[8,2,3,4,5,6,1,7,9]	eliminate	7 and 9 order swapped
[8,2,3,4,5,6,1,9,5]	eliminate	duplicate 5, missing 7

OX vs PMX in one sentence

PMX says: "repair conflicts by mapping." OX says: "preserve relative order from the other parent."

20. CX option-elimination lab

CX elimination ladder

1. Check valid permutation.
2. Build cycles; do not guess.
3. Every position belongs to a cycle.
4. A child must take whole cycles from one parent, not individual positions randomly.
5. If an option mixes a cycle, eliminate.

CX worked MCQ with options

Parents:

P1 = [1,2,3,4,5,6,7,8]
P2 = [3,7,5,1,6,8,2,4]

Cycles:

Cycle 1 = {1,3,5,6,8,4}
Cycle 2 = {2,7}

Children:

C1 = [1,7,3,4,5,6,2,8]
C2 = [3,2,5,1,6,8,7,4]

Fake options:

Candidate	Decision	Reason
[1,7,3,4,5,6,2,8]	keep	exact C1
[1,2,3,1,5,6,2,8]	eliminate	duplicate 1 and 2, invalid permutation
[3,2,5,1,6,8,7,4]	keep	exact C2
[1,7,5,4,5,8,2,8]	eliminate	invalid and mixed cycles

CX corrupted-key defense

If the key says an option is correct but it violates full-cycle inheritance, treat the key as risky. In CX, the algorithm is mechanical: cycles decide the positions.

21. ECX option-elimination lab

ECX is harder because tie-breaks can vary. The exam often tests recognition more than full computation.

ECX recognition ladder

MCQ wording	Operator
common edges	ECX
adjacency relations	ECX
shared neighboring cities	ECX
edge table	ECX
relative order	OX, not ECX

absolute position	CX, not ECX
mapping conflict	PMX, not ECX

ECX validation

A valid ECX child must still be a valid permutation. If an option repeats or misses a city, eliminate immediately.

Part XII - More detailed mutation laboratories

22. Ambiguous before/after mutation cases

Some before/after patterns can be explained by more than one operation. The exam wording decides.

Example:

```
Before = [1,2,3,4,5]
After  = [1,4,3,2,5]
```

This could be:

- swap positions 2 and 4, or
- inversion positions 2..4, because [2,3,4] reversed is [4,3,2].

If the MCQ asks "which mutation could have produced this," both may be possible. If it asks "following inversion with positions 2 and 4," compute inversion. If it asks "which mutation changed exactly two positions," choose swap.

23. Mutation by representation: exhaustive summary

Representation	Mutation	Input	Output	How to validate
Binary	bit flip	bit string	bit string	all values still 0/1
Integer	creep	integer vector	integer vector	value changed slightly and within bounds
Integer	random reset	integer vector	integer vector	replaced value valid in domain
Real	uniform	real vector	real vector	value inside bounds
Real	non-uniform	real vector + generation	real vector	step size depends on generation
Real	Gaussian	real vector + sigma/noise	real vector	same dimension, within bounds
Permutation	swap	permutation	permutation	two positions exchanged
Permutation	insertion	permutation	permutation	one moved, block shifted
Permutation	inversion	permutation	permutation	interval reversed
Permutation	scramble	permutation	permutation	interval shuffled, same set
ES	self-adaptive Gaussian	(x,sigma)	(x',sigma')	sigma positive, x real

24. Mutation practice - extended set

Exercise M8

Before [1,2,3,4,5,6,7], after [1,2,6,5,4,3,7]. Identify mutation and positions.

Solution: inversion positions 3..6.

Exercise M9

Before [1,2,3,4,5,6,7], after [1,5,3,2,4,6,7]. Identify likely mutation.

Solution: scramble positions 2..5 because values {2,3,4,5} are shuffled. It is not simple inversion because [2,3,4,5] reversed would [5,4,3,2].

Exercise M10

Before [1,2,3,4,5], after [1,3,4,5,2]. Identify mutation.

Solution: insertion; value 2 moved from position 2 to position 5.

Exercise M11

Before [9,4,7,2], after [9,4,8,2]. Identify mutation.

Solution: creep/fluaj at position 3 (+1).

Exercise M12

Before [9,4,7,2], after [9,4,1,2]. Identify mutation.

Solution: random reset at position 3, unless domain/wording says large creep is allowed.

Exercise M13

Before [0.4,0.8,0.1], after [0.4,0.37,0.1]. Mutation?

Solution: real uniform mutation if the gene was replaced by a random value from interval. If noise/sigma is given, it may be Gaussian. Use wording.

Exercise M14

$x=[5.0]$, $\sigma=0.2$, $z=-3$. What is x' ?

Solution: $x'=5.0+0.2*(-3)=4.4$.

Part XIII - Selection advanced details and traps

25. FPS in minimization problems

FPS assumes larger fitness is better. For minimization, transform objective into fitness.

Common transformations:

$\text{fitness} = 1 / \text{cost}$	if $\text{cost} > 0$
$\text{fitness} = -\text{cost}$	if values allow comparison and selection method handles signs
$\text{fitness} = C - \text{cost}$	if C is large enough to keep positive values

Example

Costs:

A=10, B=20, C=40

Use fitness = 1/cost:

A=0.1, B=0.05, C=0.025

A has highest fitness because it has lowest cost.

26. FPS with zero or negative values

Roulette/FPS needs nonnegative probabilities. If fitness values are zero or negative, scale or shift them before using FPS.

Example:

raw scores = [-5, 0, 10]

Shift by +6:

scaled = [1,6,16]

Then compute probabilities.

27. Tournament variants

Variant	Meaning
deterministic tournament	best in sample always wins
probabilistic tournament	best wins with probability p
with replacement	same individual may appear again in later tournaments
without replacement	sample excludes already chosen within tournament

Most MCQs only require: tournament samples a subset and selects the best; it does not exactly respect the FPS probability distribution.

28. Selection practice - extended set

Exercise SEL1

Costs A=10, B=20, C=50. Use fitness=1/cost. Which has highest selection probability?

Solution: A.

Exercise SEL2

Fitness A=0, B=5, C=15. Compute probabilities.

Solution: total=20. A=0, B=0.25, C=0.75.

Exercise SEL3

Fitness A=10, B=90. What is the FPS risk?

Solution: B dominates selection, causing premature convergence.

Exercise SEL4

Which selection method uses multiple equally spaced pointers?

Solution: SUS.

Exercise SEL5

Which selection method samples k individuals and picks the best?

Solution: tournament.

Part XIV - Survivor selection extended laboratories

29. Full replacement examples

Example: generational vs elitist generational

Current:

A=100, B=70, C=60, D=50

Offspring:

E=90, F=80, G=40, H=30

Generational:

Next = [E, F, G, H]

Elitist generational with $k=1$:

Next = [A, E, F, G] if filling with first three offspring

Best 4 from combined quality-based:

Next = [A, E, F, B]

These are different rules. Read the wording.

30. Age vs quality examples

Current:

Individual	Fitness	Age
A	100	9
B	70	2
C	50	1
D	20	1

If age-based removes oldest, A is removed. If quality-based removes worst, D is removed. Therefore age-based does not guarantee quality improvement.

31. Survivor selection practice - extended set

Exercise SUR1

Current A=90 age 10, B=50 age 1, C=40 age 1. New offspring D=60. If replacing oldest, who is removed and what is next population?

Solution: Remove A. Next [B,C,D].

Exercise SUR2

Same data, if replacing worst quality, who is removed?

Solution: C=40. Next [A,B,D].

Exercise SUR3

Current [80,70,60], offspring [90,50,40]. Best 3 combined?

Solution: [90,80,70].

Exercise SUR4

Current [80,70,60], offspring [90,50,40]. Generational replacement?

Solution: [90,50,40].

Part XV - Final comprehensive exam simulation

32. Mixed MCQ simulation with answer key

Q1

Which representation is most appropriate for TSP?

Answer: permutation.

Q2

Which crossover preserves relative order?

Answer: OX/OCX.

Q3

Which crossover preserves absolute positions using cycles?

Answer: CX.

Q4

Which crossover preserves common edges?

Answer: ECX.

Q5

Which crossover uses conflict mapping?

Answer: PMX.

Q6

P1=[1,2,3,4], P2=[4,3,2,1], one-point cut after 2. Raw C1?

Answer: [1,2,2,1]. Invalid as permutation.

Q7

Original [1,2,3,4,5], mutated [1,2,5,4,3]. Mutation?

Answer: inversion positions 3..5.

Q8

Original [1,2,3,4,5], mutated [1,4,2,3,5]. Mutation?

Answer: insertion: value 4 moved to position 2, shifting 2 and 3 right. Also could be other sequences depending operations; MCQ wording decides.

Q9

Original [1,2,3,4,5], mutated [1,3,4,2,5]. Mutation?

Answer: insertion: value 2 moved after 4.

Q10

Original [1,2,3,4,5], mutated [1,4,3,2,5]. Mutation?

Answer: ambiguous swap 2 and 4, or inversion 2..4. Use wording.

Q11

Fitness A=10, B=20, C=70. FPS probabilities?

Answer: A=0.10, B=0.20, C=0.70.

Q12

SUS N=5, start=0.03. Pointers?

Answer: 0.03, 0.23, 0.43, 0.63, 0.83.

Q13

Tournament sample has fitness [12,44,20]. Winner?

Answer: 44.

Q14

ES mutation probability?

Answer: 1.

Q15

ES individual ([2,5], sigma=0.2), z=[1,-1]. Mutated x?

Answer: [2.2,4.8].

Q16

(mu,lambda) selection uses which pool?

Answer: offspring only.

Q17

(mu+lambda) selection uses which pool?

Answer: parents + offspring.

Q18

Which selection method is closest to target probability distribution?

Answer: SUS.

Q19

Which mutation is a small integer change?

Answer: creep/fluaj.

Q20

Which mutation shuffles values in an interval?

Answer: scramble/amestec.

33. Final command checklist for manual application

Before choosing an MCQ answer, ask:

1. Is the chromosome binary, integer, real, or permutation?
2. Is the operation mutation or recombination?
3. Does the operation use one chromosome or two parents?
4. If permutation, is the output a valid permutation?
5. If PMX, did I map and repair conflicts?
6. If OX, did I preserve relative order?
7. If CX, did I use whole cycles?
8. If ECX, did I preserve edges/adjacency?
9. If mutation, did I diagnose before/after correctly?
10. If selection, did I distinguish parent vs survivor?
11. If ES, did I remember mutation probability 1?
12. If ES selection, did I choose offspring-only or parents+offspring correctly?
13. If the key looks weird, is this a known exam-risk pattern?

34. What counts as complete mastery

You have mastered this topic when you can do all of this without notes:

- list valid operations for binary, integer, real, and permutation;
- apply one-point, multi-point, uniform, and arithmetic crossover;
- apply PMX and compute both children;
- apply OX/OCX and compute both children;
- apply CX and compute cycles;
- identify ECX and build a basic edge table;

- identify every mutation from before/after arrays;
- compute FPS, roulette, SUS, and tournament outcomes;
- compute survivor selection under generational, steady-state, elitism, age, quality, (μ, λ) , $(\mu + \lambda)$;
- distinguish GA and ES for every component;
- explain why fake MCQ options are false.